

Neural Network Architectures

From Recurrence and Latent Codes to Attention and Graphs

Extended Technical Study Guide

Assumed background. You know the basic neural-network ingredients: affine layers, nonlinear activations, gradient descent/backpropagation, losses, batches, and train/validation/test splits. This guide focuses on the architectural ideas that distinguish RNNs, LSTMs, GRUs, encoder-decoder systems, autoencoders, Transformers, GNNs, and GATs.

Learning path

1. Recurrent neural networks (RNNs): state, unrolling, and backpropagation through time
2. LSTMs and GRUs: learned gates for memory and gradient flow
3. Encoders and decoders: the general representation-to-output pattern
4. Autoencoders: representation learning by reconstruction, including denoising and VAEs
5. Attention and Transformers: content-dependent routing without recurrence
6. Graph neural networks (GNNs): message passing on relational data
7. Graph attention networks (GATs): learned weighting of graph neighbors
8. Comparison, hybrids, and model-selection intuition

Key point

The central question throughout the guide is: *how does information move?* RNNs move information through time via a recurrent state; autoencoders compress and reconstruct through a latent code; Transformers route information globally through attention; GNNs move information along graph edges; GATs learn which neighbors should matter most.

Contents

1	Notation and the Big Picture	4
1.1	Notation used throughout	4
1.2	One-page conceptual map	4
2	Recurrent Neural Networks (RNNs)	5
2.1	Why recurrence?	5
2.2	What the hidden state represents	5
2.3	Worked forward pass	6
2.4	Backpropagation through time (BPTT)	6
2.5	Vanishing and exploding gradients	7
2.6	When a vanilla RNN is still useful	7
3	Long Short-Term Memory (LSTM)	8
3.1	The design idea	8
3.2	Core equations	8
3.3	Why the cell state helps gradient flow	8
3.4	Worked gate update	9
3.5	Parameter count	9
3.6	Practical behavior	9
4	Gated Recurrent Unit (GRU)	11
4.1	A more compact gated recurrent cell	11
4.2	Interpreting the gates	11
4.3	GRU versus LSTM	11
5	Encoders and Decoders	12
5.1	A role, not a specific layer type	12
5.2	Classic recurrent sequence-to-sequence modeling	12
5.3	Teacher forcing and autoregressive inference	12
5.4	The fixed-context bottleneck	12
5.5	Encoder-only, decoder-only, and encoder-decoder	13
6	Autoencoders	14
6.1	Reconstruction as a representation-learning objective	14
6.2	Reconstruction losses	14
6.3	The bottleneck principle	14
6.4	Denoising autoencoders	15
6.5	Sparse autoencoders	15
6.6	Autoencoders for anomaly detection	15
6.7	Variational autoencoders (VAEs)	16
6.8	What autoencoders are good for	16
7	Attention: Queries, Keys, and Values	17
7.1	Content-based retrieval	17
7.2	Why scale by $\sqrt{d_k}$?	17
7.3	Numerical attention example	17
7.4	Self-attention and cross-attention	17
7.5	Multi-head attention	18

8	Transformers	19
8.1	From attention to a full architecture	19
8.2	Positional information	19
8.3	Transformer encoder block	19
8.4	Transformer decoder block	20
8.5	Shape bookkeeping example	20
8.6	Why Transformers train efficiently	21
8.7	Complexity comparison	21
8.8	Encoder-only, decoder-only, encoder-decoder Transformers	21
8.9	Training and inference	21
8.10	Strengths and limitations	22
9	Graph Neural Networks (GNNs)	23
9.1	Why graphs require a different inductive bias	23
9.2	The message-passing template	23
9.3	Graph Convolutional Network (GCN)	23
9.4	Worked neighborhood aggregation	23
9.5	Permutation equivariance	24
9.6	Graph learning tasks	24
9.7	Over-smoothing	24
9.8	Over-squashing and long-range graph dependencies	25
9.9	Homophily and heterophily	25
10	Graph Attention Networks (GATs)	26
10.1	Learning which neighbors matter	26
10.2	Numerical softmax example	26
10.3	Multi-head graph attention	27
10.4	GAT versus Transformer attention	27
10.5	Interpretability caveat	27
11	Putting the Architectures Together	28
11.1	Comparison table	28
11.2	Information-flow comparison	28
11.3	Choosing by problem structure	28
11.4	Data regime matters	29
11.5	Hybrids are common	29
11.6	Training pathologies at a glance	29
12	Deeper Connections Between the Models	30
12.1	Gates and attention are both selective routing	30
12.2	Residual connections resemble memory highways	30
12.3	Attention as dynamic message passing	30
12.4	Autoencoders and encoders are not synonymous	30
13	Practical Design Checklist	31
13.1	Before choosing an architecture	31
13.2	Sequence-model checklist	31
13.3	Graph-model checklist	31
13.4	Evaluation discipline	31
14	Compact Mathematical Cheat Sheet	32

Selected References and Further Reading**33**

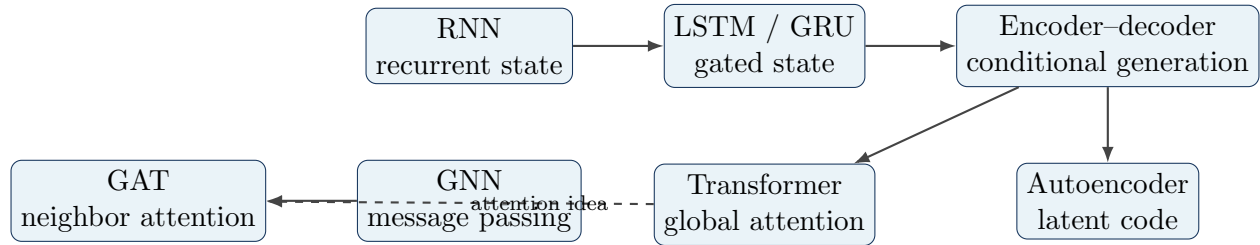
1 Notation and the Big Picture

A useful way to compare architectures is to separate three ideas: the **data structure**, the **information-routing rule**, and the **prediction objective**. A model can use the same loss as another model while having a very different routing rule.

1.1 Notation used throughout

Symbol	Meaning
$x_t \in \mathbb{R}^{d_x}$	input vector at sequence step t
$h_t \in \mathbb{R}^{d_h}$	recurrent hidden state
$c_t \in \mathbb{R}^{d_h}$	LSTM cell state
T	input sequence length
$X \in \mathbb{R}^{T \times d}$	matrix of token or time-step representations
Q, K, V	query, key, and value matrices in attention
$G = (V, E)$	graph with node set V and edge set E
A	graph adjacency matrix
$H^{(\ell)}$	node-feature matrix at GNN layer ℓ
$\odot$	elementwise multiplication
$[a; b]$ or $a \ b$	vector concatenation

1.2 One-page conceptual map



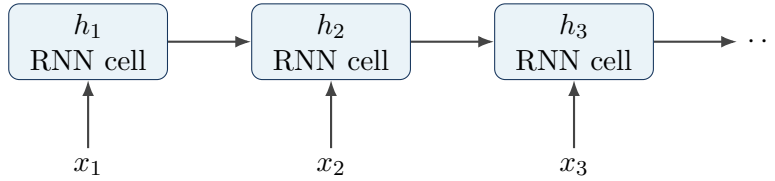
Intuition

There is no strict hierarchy in which one model “replaces” all earlier ones. A Transformer often dominates for large-scale text, but an LSTM can still be excellent for modest sequential datasets; a GNN is natural when the input itself is a graph; an autoencoder solves a different representation-learning problem entirely.

2 Recurrent Neural Networks (RNNs)

2.1 Why recurrence?

A standard feed-forward network assumes a fixed-size input and does not, by itself, preserve a state from one element of a sequence to the next. An RNN introduces an explicit hidden state h_t that is updated repeatedly with the *same parameters*.



Core equations

A vanilla RNN commonly uses

$$a_t = W_x x_t + W_h h_{t-1} + b_h, \quad (1)$$

$$h_t = \phi(a_t), \quad (2)$$

$$y_t = W_y h_t + b_y, \quad (3)$$

where ϕ is typically tanh or, less commonly in classic RNNs, ReLU.

If $x_t \in \mathbb{R}^{d_x}$ and $h_t \in \mathbb{R}^{d_h}$, then $W_x \in \mathbb{R}^{d_h \times d_x}$ and $W_h \in \mathbb{R}^{d_h \times d_h}$. The recurrent part therefore has

$$d_h d_x + d_h^2 + d_h$$

trainable parameters before the output layer is counted.

2.2 What the hidden state represents

The hidden state is a learned summary of previous inputs:

$$h_t = F(x_t, h_{t-1}) = F(x_t, F(x_{t-1}, \dots)).$$

It is not guaranteed to be a human-interpretable memory. Rather, gradient-based training shapes it to retain the aspects of history that help the loss.

Common sequence mappings include:

- **many-to-one**: a whole sequence maps to one output, e.g. sequence classification;
- **many-to-many, aligned**: one output per input step, e.g. tagging or time-series filtering;
- **many-to-many, unaligned**: an input sequence maps to an output sequence of a different length, usually through an encoder-decoder architecture.

2.3 Worked forward pass

Worked example

Consider a scalar RNN with $h_0 = 0$, $w_x = 0.8$, $w_h = 0.6$, $b = 0$, $\phi = \tanh$, and inputs

$$x_1 = 1, \quad x_2 = 0.5, \quad x_3 = -0.5.$$

Then

$$\begin{aligned} h_1 &= \tanh(0.8 \cdot 1 + 0.6 \cdot 0) \approx 0.664, \\ h_2 &= \tanh(0.8 \cdot 0.5 + 0.6 \cdot 0.664) \approx 0.663, \\ h_3 &= \tanh(0.8 \cdot (-0.5) + 0.6 \cdot 0.663) \approx -0.002. \end{aligned}$$

The third state depends on all three inputs, not only on x_3 . The network has learned a stateful dynamical system.

2.4 Backpropagation through time (BPTT)

Training an RNN requires differentiating through the unrolled recurrent computation. Suppose a loss at time T depends on h_T . For $k < T$,

$$\frac{\partial \mathcal{L}}{\partial h_k} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}}, \quad (4)$$

$$\frac{\partial h_j}{\partial h_{j-1}} = \text{diag}(\phi'(a_j)) W_h. \quad (5)$$

The gradient therefore contains a *product of many Jacobians*. Products with norms mostly below one shrink; products with norms above one can grow rapidly.

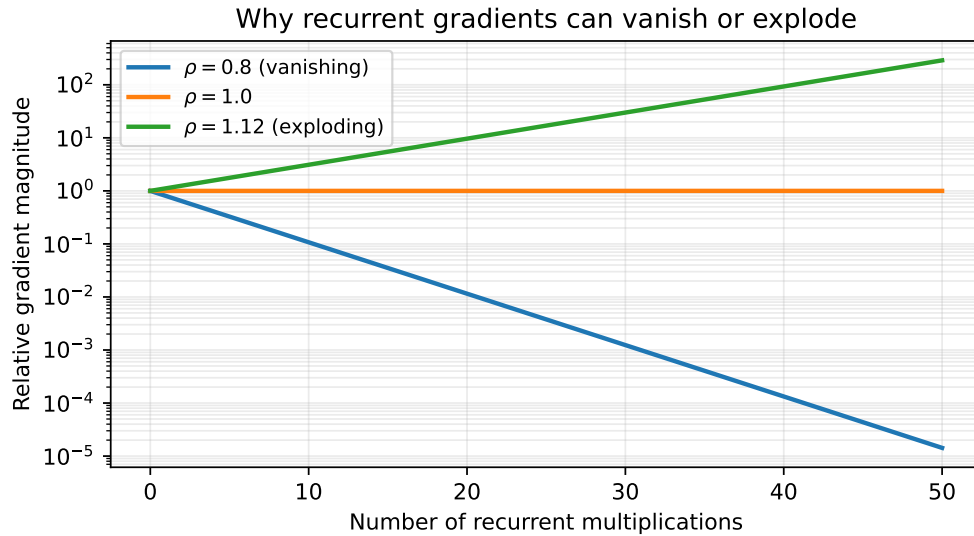


Figure 1: A scalar analogy for recurrent gradient propagation. The exact behavior of a real RNN depends on matrices and activation derivatives, but the multiplicative mechanism is the same.

2.5 Vanishing and exploding gradients

For $\tanh$, $|\phi'| \leq 1$, so repeated multiplication can quickly reduce sensitivity to distant states. This is the **vanishing-gradient** problem and is one reason vanilla RNNs struggle with long-range dependencies. Conversely, a recurrent matrix with sufficiently large effective singular values can cause exploding gradients.

Practical responses include:

- gradient clipping for exploding gradients;
- careful initialization and normalization;
- truncated BPTT for long streams;
- gated architectures such as LSTM or GRU;
- attention-based architectures when global dependencies are important.

Common pitfall

“An RNN has memory” does not imply it can remember arbitrarily long histories. The memory length is an emergent consequence of the dynamics and training. Vanilla RNNs often forget distant information even when the sequence is technically available.

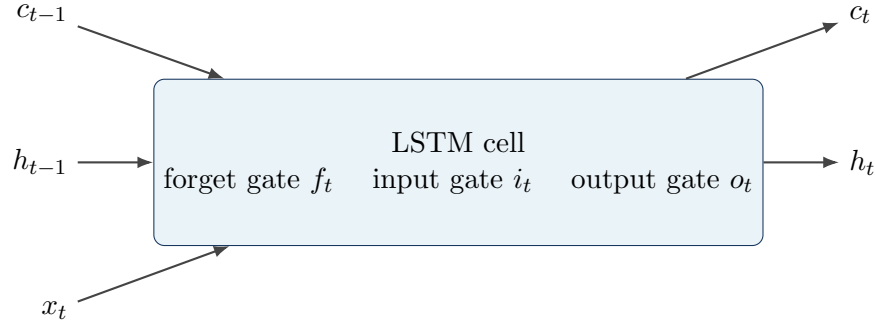
2.6 When a vanilla RNN is still useful

A vanilla RNN is valuable as a conceptual baseline and can be adequate when dependencies are short, datasets are modest, latency is important, or the sequential inductive bias is more valuable than global context. It is also useful for understanding why gated recurrence and attention were developed.

3 Long Short-Term Memory (LSTM)

3.1 The design idea

An LSTM introduces a cell state c_t with an additive update. Instead of forcing all information through repeated nonlinear transformations of one hidden state, it creates a path on which information can persist and be selectively erased or written.



3.2 Core equations

For concatenated input $u_t = [h_{t-1}; x_t]$,

Core equations

$$f_t = \text{sigmoid}(W_f u_t + b_f) \quad \text{forget gate,} \quad (6)$$

$$i_t = \text{sigmoid}(W_i u_t + b_i) \quad \text{input gate,} \quad (7)$$

$$\tilde{c}_t = \tanh(W_c u_t + b_c) \quad \text{candidate content,} \quad (8)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{cell update,} \quad (9)$$

$$o_t = \text{sigmoid}(W_o u_t + b_o) \quad \text{output gate,} \quad (10)$$

$$h_t = o_t \odot \tanh(c_t). \quad (11)$$

The gates lie in $(0, 1)$ elementwise. Each hidden dimension can therefore learn a different timescale.

3.3 Why the cell state helps gradient flow

Ignoring indirect paths through the gates for a moment,

$$\frac{\partial c_t}{\partial c_{t-1}} \approx f_t.$$

Over k steps,

$$\frac{\partial c_t}{\partial c_{t-k}} \approx \prod_{j=t-k+1}^t f_j.$$

If forget gates remain close to one, the derivative can stay large enough to preserve useful long-range credit assignment.

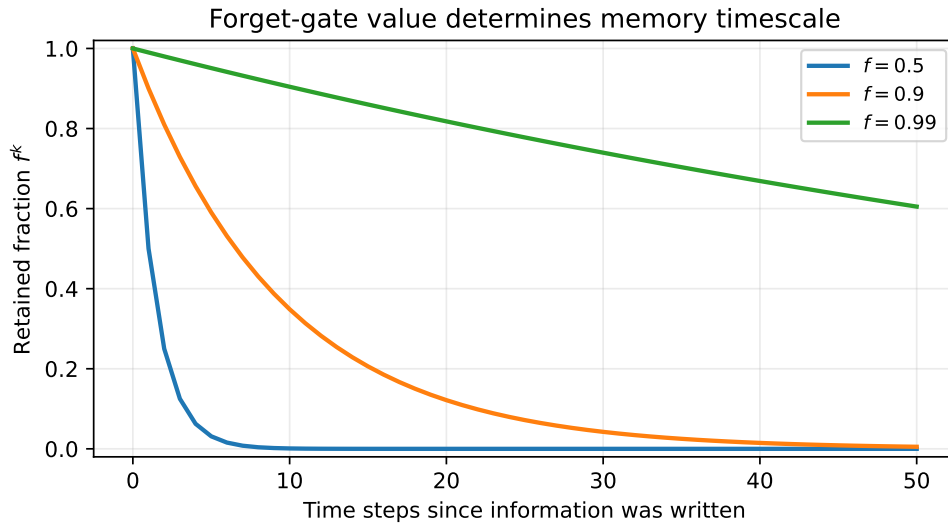


Figure 2: If the forget gate is approximately constant, information retained after k steps is proportional to f^k . A gate of 0.99 supports a much longer effective memory than a gate of 0.5.

3.4 Worked gate update

Worked example

For one scalar LSTM unit, suppose

$$c_{t-1} = 0.4, \quad f_t = 0.9, \quad i_t = 0.3, \quad \tilde{c}_t = 0.5, \quad o_t = 0.8.$$

Then

$$c_t = 0.9(0.4) + 0.3(0.5) = 0.51,$$

and

$$h_t = 0.8 \tanh(0.51) \approx 0.376.$$

The cell preserved most of the old memory (0.36) and added a smaller amount of new candidate content (0.15).

3.5 Parameter count

With input width d_x and hidden width d_h , each of the four gate/candidate transformations maps a $(d_x + d_h)$ vector to d_h . Therefore an LSTM has roughly

$$4d_h(d_x + d_h + 1)$$

recurrent parameters. This is about four times the affine-parameter cost of a comparable vanilla RNN cell.

3.6 Practical behavior

LSTMs work well when sequences are moderate in length and the data volume does not justify a very large Transformer. They are common in time series, biosignals, forecasting, speech pipelines, and embedded or streaming settings.

Useful variants include bidirectional LSTMs, stacked LSTMs, peephole connections, and LSTMs with attention. A bidirectional LSTM uses past and future context, so it is inappropriate when future inputs would constitute leakage at inference time.

Key point

The LSTM is not “a network that remembers everything.” It is a network that learns *when to retain, overwrite, and expose* information through differentiable gates.

4 Gated Recurrent Unit (GRU)

4.1 A more compact gated recurrent cell

The GRU merges the LSTM's cell and hidden state into a single state and typically uses two gates: an **update gate** z_t and a **reset gate** r_t .

Core equations

One common convention is

$$z_t = \text{sigmoid}(W_z x_t + U_z h_{t-1} + b_z), \quad (12)$$

$$r_t = \text{sigmoid}(W_r x_t + U_r h_{t-1} + b_r), \quad (13)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \quad (14)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (15)$$

Some libraries swap the interpretation of z_t in the final interpolation; always check the implementation convention.

4.2 Interpreting the gates

- z_t controls how aggressively the state moves toward the newly proposed candidate.
- r_t controls how much previous-state information is used when building that candidate.
- The interpolation in the last equation creates an additive path similar in spirit to the LSTM's cell-state update.

Worked example

Suppose $h_{t-1} = 0.6$, $z_t = 0.25$, $r_t = 0.8$, and the candidate has already been computed as $\tilde{h}_t = -0.2$. Then

$$h_t = 0.75(0.6) + 0.25(-0.2) = 0.40.$$

Because the update gate is small, the GRU keeps most of the old state.

4.3 GRU versus LSTM

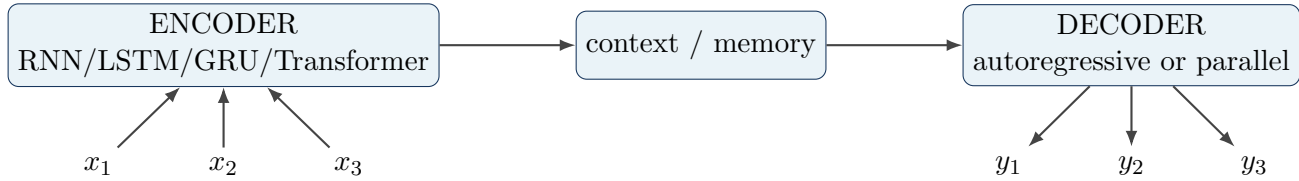
	LSTM	GRU
State	hidden state plus separate cell state	one main hidden state
Main gates	forget, input, output	update, reset
Parameter count	larger	smaller
Interpretability	more explicit memory path	simpler state interpolation
Training speed	often somewhat slower	often somewhat faster
Accuracy	task-dependent	task-dependent

There is no theorem saying one must dominate. When both are plausible, empirical validation is the right decision rule.

5 Encoders and Decoders

5.1 A role, not a specific layer type

An **encoder** transforms an input into a representation that is convenient for the task. A **decoder** transforms that representation into an output. Either side may be an RNN, LSTM, GRU, Transformer, CNN, GNN, or another differentiable architecture.



5.2 Classic recurrent sequence-to-sequence modeling

Let $x_{1:T}$ be the input and $y_{1:U}$ the output. The encoder produces a context representation $c = \text{Enc}(x_{1:T})$. The decoder defines a conditional distribution

$$p(y_{1:U} \mid x_{1:T}) = \prod_{u=1}^U p(y_u \mid y_{<u}, c). \quad (16)$$

For discrete outputs, the training objective is usually token-level cross-entropy (negative conditional log-likelihood):

$$\mathcal{L} = - \sum_{u=1}^U \log p(y_u^* \mid y_{<u}^*, c).$$

5.3 Teacher forcing and autoregressive inference

During training, the decoder can receive the *true* previous token y_{u-1}^* . This is teacher forcing. At inference time, that token is unavailable, so the decoder receives its own previous prediction $\hat{y}_{u-1}$.

This creates **exposure bias**: the model is optimized under cleaner histories than it sees during free-running generation. Scheduled sampling and sequence-level objectives have been proposed to reduce the mismatch, but standard teacher forcing remains common.

Worked example

In translation, an English encoder might process “the cat sleeps” into a set of hidden states. A French decoder generates “le”, then “chat”, then “dort”. The decoder’s second decision depends on the encoded source and on the already generated token “le”.

5.4 The fixed-context bottleneck

Early recurrent seq2seq systems often compressed the entire input into the encoder’s final state. For long sequences, forcing all source information through one fixed-width vector becomes difficult. Attention solves this by exposing the decoder to *all* encoder states.

A classic additive attention mechanism uses

$$e_{u,t} = v^\top \tanh(W_s s_{u-1} + W_h h_t), \quad (17)$$

$$\alpha_{u,t} = \frac{\exp(e_{u,t})}{\sum_{j=1}^T \exp(e_{u,j})}, \quad (18)$$

$$c_u = \sum_{t=1}^T \alpha_{u,t} h_t. \quad (19)$$

Now the context vector c_u changes for every decoder step u .

Key point

Attention first became important as a way for a decoder to retrieve the relevant parts of an encoder's sequence. The Transformer generalizes the same idea: representations repeatedly retrieve information from other representations using learned queries and keys.

5.5 Encoder-only, decoder-only, and encoder-decoder

Form	Information flow	Typical uses
Encoder-only	all input elements contextualize each other	classification, embeddings, tagging, retrieval
Decoder-only	each position accesses only allowed prior context	autoregressive generation, next-token prediction
Encoder-decoder	encoder builds source memory; decoder generates while attending to it	translation, summarization, structured transduction

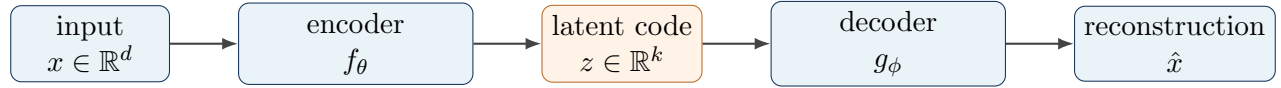
6 Autoencoders

6.1 Reconstruction as a representation-learning objective

An autoencoder is a specific encoder-decoder system in which the target is usually the input itself:

$$z = f_{\theta}(x), \quad \hat{x} = g_{\phi}(z).$$

The latent representation z is useful only if the training setup prevents the network from learning a trivial identity mapping.



6.2 Reconstruction losses

For continuous inputs, mean-squared error is common:

$$\mathcal{L}_{\text{rec}}(x, \hat{x}) = \frac{1}{d} \|x - \hat{x}\|_2^2.$$

For binary or Bernoulli-modeled inputs, a componentwise binary cross-entropy can be appropriate. The loss should match the data-generating assumptions and scaling.

6.3 The bottleneck principle

If $k < d$, the autoencoder is **undercomplete**: the latent code has lower dimension than the input. It must prioritize information that supports reconstruction. However, dimensionality alone is not sufficient for useful semantics; a nonlinear network can still memorize finite training examples if capacity is excessive.

A linear autoencoder with an undercomplete bottleneck and squared reconstruction loss learns a subspace related to the principal subspace of PCA. Nonlinear autoencoders generalize this idea to curved manifolds.

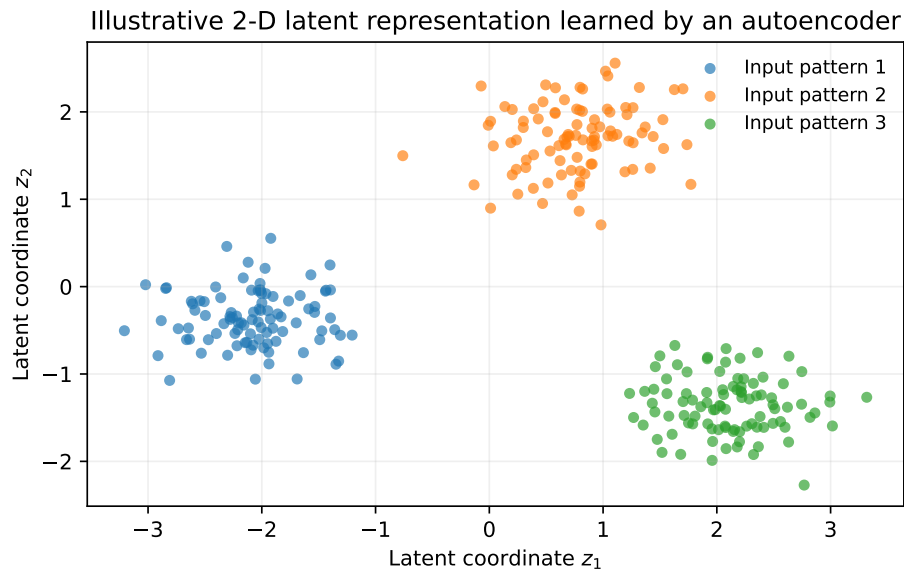


Figure 3: Illustrative latent representation. An autoencoder is not directly told about classes here; structure can emerge because similar inputs require similar reconstruction factors.

6.4 Denoising autoencoders

A denoising autoencoder corrupts the input, but asks the model to reconstruct the original clean sample:

$$\tilde{x} \sim q(\tilde{x} | x), \quad (20)$$

$$z = f_{\theta}(\tilde{x}), \quad (21)$$

$$\mathcal{L} = \|x - g_{\phi}(z)\|^2. \quad (22)$$

Because copying corrupted input is not sufficient, the representation is encouraged to capture stable structure.

6.5 Sparse autoencoders

A sparse autoencoder can have a large latent layer while penalizing frequent activation. If $\hat{\rho}_j$ is the average activation of latent unit j and ρ is a small target activity, a common penalty is

$$\Omega_{\text{sparse}} = \sum_j D_{\text{KL}}(\text{Bern}(\rho) \parallel \text{Bern}(\hat{\rho}_j)).$$

The full objective becomes reconstruction error plus $\beta\Omega_{\text{sparse}}$.

6.6 Autoencoders for anomaly detection

Train an autoencoder primarily on normal data. At inference, compute a reconstruction score such as

$$s(x) = \|x - \hat{x}\|_2^2.$$

Samples with unusually large scores may be anomalous. This works only if the model reconstructs normal structure substantially better than out-of-distribution structure; an overly expressive autoencoder can reconstruct anomalies too well.

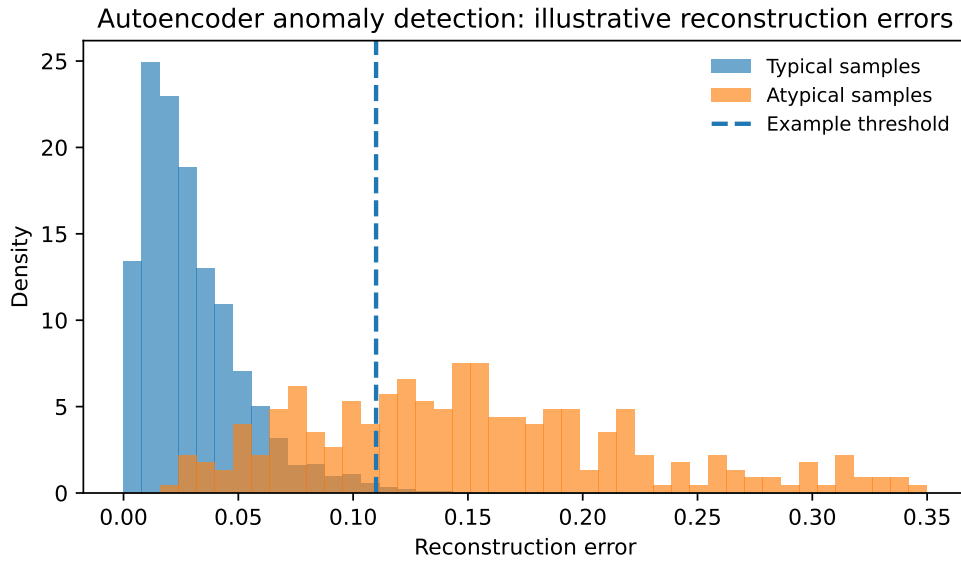


Figure 4: Illustrative anomaly-detection use. A threshold can be selected using validation data, desired false-positive rate, or downstream cost.

6.7 Variational autoencoders (VAEs)

A VAE is probabilistic. The encoder parameterizes an approximate posterior distribution rather than one deterministic point:

$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x))).$$

A simple prior is $p(z) = \mathcal{N}(0, I)$. The evidence lower bound (ELBO) is

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] - D_{\text{KL}}(q_{\phi}(z | x) || p(z)). \quad (23)$$

Training minimizes the negative ELBO. The first term rewards reconstruction or likelihood; the KL term regularizes the posterior toward the prior.

To differentiate through sampling, use the reparameterization trick:

$$\epsilon \sim \mathcal{N}(0, I), \quad z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon.$$

Common pitfall

A VAE and a deterministic autoencoder should not be conflated. A deterministic autoencoder learns a code useful for reconstruction. A VAE learns a latent *distribution* and a generative model, with an explicit regularization toward a prior.

6.8 What autoencoders are good for

Compression, representation learning, denoising, nonlinear dimensionality reduction, anomaly scoring, pretraining, and generative modeling (for VAEs) are common uses. But if the downstream task has labels, supervised representation learning can be more directly aligned with the final objective.

7 Attention: Queries, Keys, and Values

7.1 Content-based retrieval

Attention lets one representation retrieve a weighted combination of other representations. Instead of passing information only through adjacent time steps, a query can directly access any allowed key/value position.

Given $X \in \mathbb{R}^{T \times d_{\text{model}}}$,

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

The basic scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (24)$$

The softmax is applied row-wise.

7.2 Why scale by $\sqrt{d_k}$?

If query/key components are independent with variance near one, a dot product of d_k components has variance that grows with d_k . Large logits push the softmax into saturated regions. Dividing by $\sqrt{d_k}$ keeps score magnitudes more stable.

7.3 Numerical attention example

Worked example

Take one query and three key/value pairs:

$$q = [1, 0], \quad k_1 = [1, 0], \quad k_2 = [0, 1], \quad k_3 = [1, 1],$$

with $d_k = 2$. The scaled scores are

$$\frac{qK^\top}{\sqrt{2}} = [0.707, 0, 0.707].$$

Their softmax is approximately

$$\alpha = [0.401, 0.198, 0.401].$$

If

$$v_1 = [1, 0], \quad v_2 = [0, 2], \quad v_3 = [1, 1],$$

then the attention output is

$$0.401v_1 + 0.198v_2 + 0.401v_3 \approx [0.802, 0.797].$$

The output is a learned, content-dependent weighted mixture of values.

7.4 Self-attention and cross-attention

In **self-attention**, Q, K, V are all derived from the same sequence. In **cross-attention**, the queries come from one sequence (often the decoder) while keys and values come from another (often encoder memory).

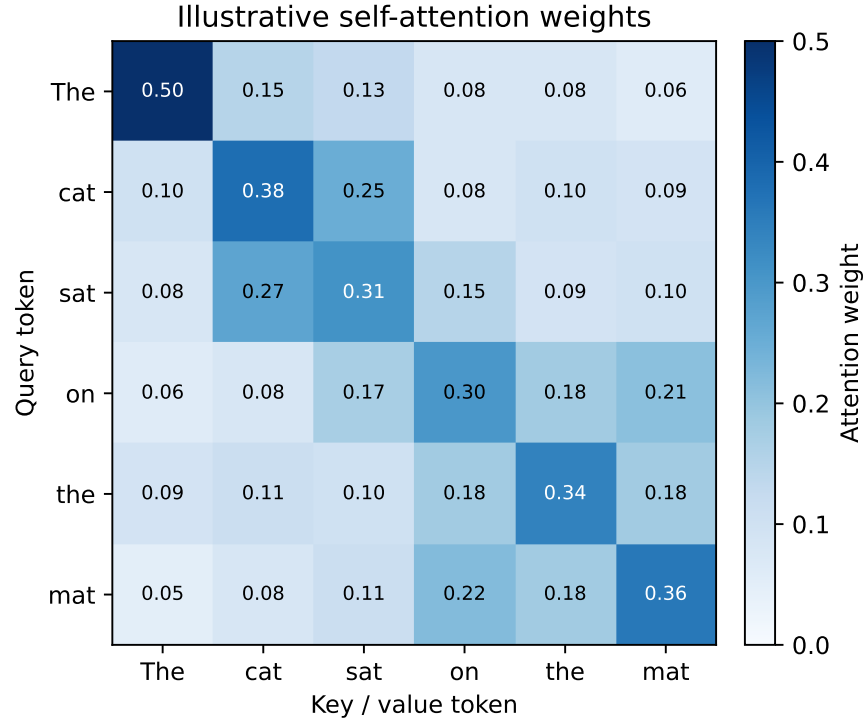


Figure 5: Illustrative self-attention matrix. Each row sums to one and shows where a query position retrieves information. Real learned patterns vary by layer, head, model, and input.

7.5 Multi-head attention

One attention pattern may be insufficient. Multi-head attention uses H independently projected subspaces:

$$\text{head}_h = \text{Attention}(QW_Q^{(h)}, KW_K^{(h)}, VW_V^{(h)}), \quad (25)$$

$$\text{MHA}(Q, K, V) = [\text{head}_1; \dots; \text{head}_H]W_O. \quad (26)$$

Different heads can specialize in different relationships, although individual-head interpretations should be treated cautiously.

8 Transformers

8.1 From attention to a full architecture

A Transformer stacks attention and position-wise feed-forward blocks with residual connections and normalization. Unlike an RNN, it does not need to process tokens sequentially during the main representation-building pass.

8.2 Positional information

Self-attention alone is permutation-equivariant: without a positional signal, swapping input positions swaps outputs in the same way. Sequence models therefore inject position information.

The original sinusoidal encoding uses

$$PE(pos, 2i) = \sin\left(pos/10000^{2i/d_{\text{model}}}\right), \quad (27)$$

$$PE(pos, 2i + 1) = \cos\left(pos/10000^{2i/d_{\text{model}}}\right). \quad (28)$$

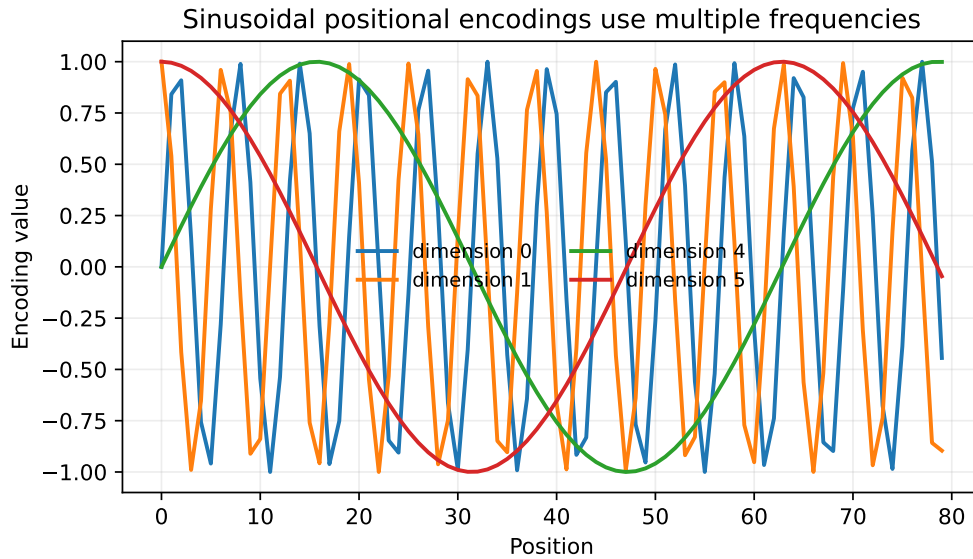


Figure 6: Sinusoidal position dimensions oscillate at different frequencies, giving the model a structured signal about position. Modern models may instead use learned, relative, or rotary position methods.

8.3 Transformer encoder block

A common schematic is

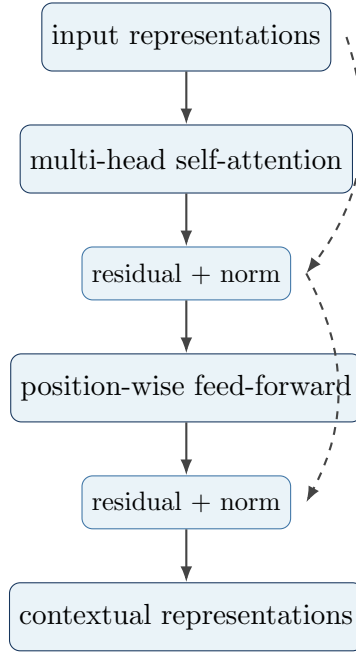
$$U = \text{Norm}(X + \text{MHA}(X, X, X)), \quad (29)$$

$$Y = \text{Norm}(U + \text{FFN}(U)), \quad (30)$$

where

$$\text{FFN}(u) = W_2 \phi(W_1 u + b_1) + b_2.$$

Modern implementations often use **pre-normalization** rather than the post-normalization form above, but the conceptual components are the same.



8.4 Transformer decoder block

An encoder-decoder Transformer decoder typically contains:

1. masked self-attention over previously generated decoder positions;
2. cross-attention whose queries come from the decoder and keys/values from the encoder;
3. a feed-forward network.

For causal language generation, use a mask M such that

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Then

$$\text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right)$$

assigns zero probability to future positions.

8.5 Shape bookkeeping example

Worked example

Suppose a batch has $B = 16$ sequences, each of length $T = 100$, with $d_{\text{model}} = 256$ and $H = 8$ attention heads. Then each head has $d_k = d_v = 32$ if dimensions are split evenly.

The hidden tensor is $16 \times 100 \times 256$. Per head, Q, K, V are $16 \times 100 \times 32$. The score tensor is $QK^\top$, giving $16 \times 100 \times 100$ per head. Across eight heads, attention stores $8 \times 16 \times 100 \times 100$ weights before implementation optimizations.

8.6 Why Transformers train efficiently

An RNN has an unavoidable sequential dependency: h_t cannot be computed before h_{t-1} . Self-attention computes all positions in a layer simultaneously, which maps well to GPUs/TPUs. The tradeoff is that dense attention compares every pair of positions.

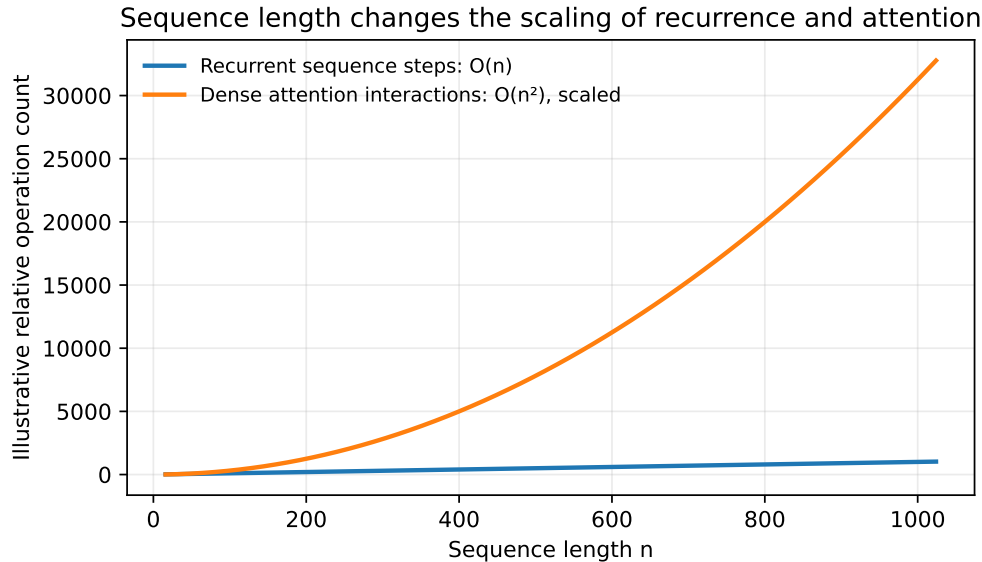


Figure 7: Illustrative scaling. Recurrent computation has a sequential chain of length n ; dense self-attention forms n^2 pairwise interactions. Constants and hardware parallelism strongly affect real runtime.

8.7 Complexity comparison

Ignoring constants and feed-forward layers, a recurrent layer is roughly $O(Td_h^2)$ in arithmetic with sequential depth $O(T)$. Dense self-attention has an $O(T^2d)$ interaction term but parallel depth $O(1)$ within a layer. For very long contexts, the T^2 memory/computation term becomes a major bottleneck, motivating sparse, local, linearized, and chunked attention methods.

8.8 Encoder-only, decoder-only, encoder-decoder Transformers

	Encoder-only	Decoder-only	Encoder-decoder
Attention visibility	bidirectional input context	causal past context	encoder bidirectional; decoder causal + cross-attention
Primary objective	representation / masked prediction	next-token prediction	conditional sequence generation
Typical tasks	classification, retrieval, tagging	open-ended generation	translation, summarization, input-conditioned generation

8.9 Training and inference

For an autoregressive decoder, training can evaluate all target positions in parallel because the entire target sequence is known and a causal mask prevents leakage. At inference, generation remains sequential

at the token level: each new token depends on previous generated tokens. Key/value caching avoids recomputing all earlier attention projections at every step.

8.10 Strengths and limitations

Strengths: long-range content-based interactions, strong parallel training, scalable pretraining, flexible encoder/decoder forms, and excellent transfer learning.

Limitations: quadratic dense-attention cost with sequence length, high data/compute demand at large scale, sensitivity to optimization and normalization details, and the need to encode position explicitly.

Key point

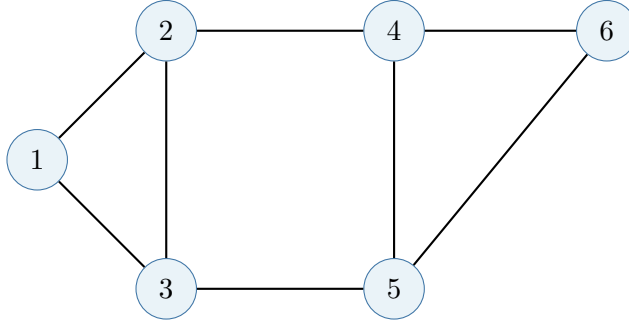
The Transformer is best understood not as “an attention layer,” but as a repeated architecture for *routing information with attention, transforming it with feed-forward sublayers, and stabilizing depth with residual paths and normalization.*

9 Graph Neural Networks (GNNs)

9.1 Why graphs require a different inductive bias

A graph represents entities and relationships. The number of neighbors varies across nodes, and node order in an input file should usually have no semantic meaning. A good graph model must therefore aggregate variable-size neighborhoods while respecting permutation symmetry.

Let $G = (V, E)$ with $n = |V|$ nodes. The adjacency matrix $A \in \{0, 1\}^{n \times n}$ satisfies $A_{ij} = 1$ when an edge connects i and j in an unweighted graph. Each node can have features $h_i^{(0)} = x_i$.



9.2 The message-passing template

Many GNNs can be described by two steps at layer ℓ :

$$m_i^{(\ell)} = \text{AGG}_{j \in \mathcal{N}(i)} \psi_\ell(h_i^{(\ell)}, h_j^{(\ell)}, e_{ij}), \quad (31)$$

$$h_i^{(\ell+1)} = \phi_\ell(h_i^{(\ell)}, m_i^{(\ell)}). \quad (32)$$

The aggregator must usually be permutation-invariant, such as sum, mean, or max. After L layers, node i can receive information from nodes within approximately L graph hops.

9.3 Graph Convolutional Network (GCN)

A widely used layer adds self-loops and symmetrically normalizes adjacency:

$$\tilde{A} = A + I, \quad (33)$$

$$\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}, \quad (34)$$

$$H^{(\ell+1)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(\ell)} W^{(\ell)} \right). \quad (35)$$

The normalization prevents high-degree nodes from dominating purely because they have more neighbors.

9.4 Worked neighborhood aggregation

Worked example

Consider three scalar node features $x_1 = 2$, $x_2 = 1$, $x_3 = 4$. Suppose node 1 is connected to nodes 2 and 3, and use a simple mean aggregator including self-information.

$$m_1 = \frac{x_1 + x_2 + x_3}{3} = \frac{7}{3} \approx 2.33.$$

If node 2 is connected only to node 1,

$$m_2 = \frac{x_2 + x_1}{2} = 1.5.$$

With a scalar learned weight $W = 0.5$ and ReLU,

$$h'_1 = \text{ReLU}(0.5 \cdot 2.33) \approx 1.17, \quad h'_2 = \text{ReLU}(0.5 \cdot 1.5) = 0.75.$$

The new node representations now contain local relational information. This is a simplified mean-message example; a normalized GCN uses the matrix formula above.

9.5 Permutation equivariance

For node-level tasks, reordering the nodes should reorder outputs in the same way rather than change their meaning. Message passing with shared parameters and symmetric neighborhood aggregation provides this property. For graph-level prediction, a final readout such as

$$h_G = \sum_{i \in V} h_i^{(L)}$$

or mean/max pooling is permutation-invariant.

9.6 Graph learning tasks

- **node classification/regression:** predict a property for each node;
- **edge or link prediction:** score whether a relation exists or what type it has;
- **graph classification/regression:** predict one target per entire graph;
- **graph generation:** generate nodes/edges or structured objects.

9.7 Over-smoothing

Repeated neighborhood averaging tends to make nearby node representations increasingly similar. Deep message-passing stacks can therefore lose discriminative node information.

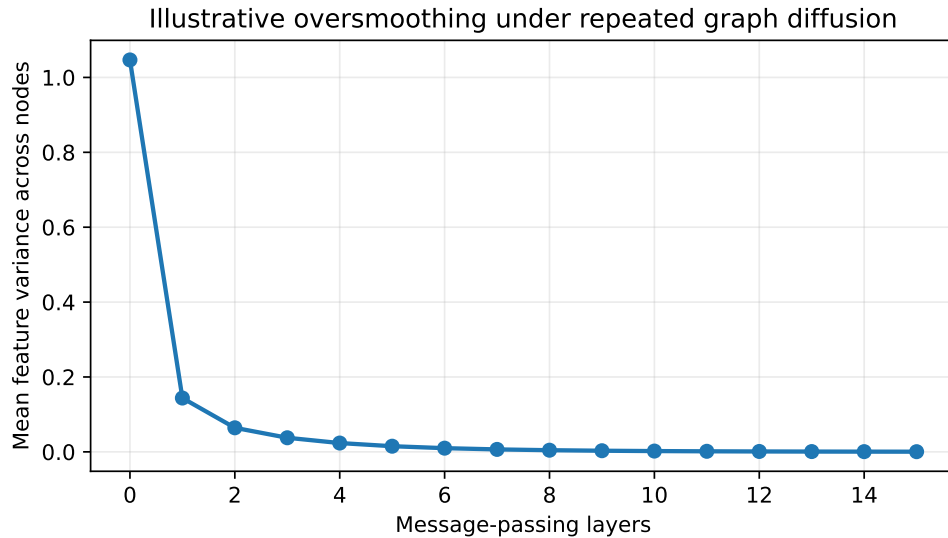


Figure 8: Illustrative graph diffusion. Repeated normalized propagation reduces node-feature variance, capturing the intuition behind over-smoothing. Real GNNs also include learned transforms, nonlinearities, residuals, and normalization.

9.8 Over-squashing and long-range graph dependencies

A different problem is **over-squashing**: information from an exponentially growing number of distant nodes may need to pass through narrow graph bottlenecks into a fixed-width vector. Even if over-smoothing is controlled, long-range information can be difficult to compress through limited message-passing channels.

9.9 Homophily and heterophily

Classical GNN intuition often assumes connected nodes are similar (homophily). In heterophilous graphs, neighbors may systematically differ, so naive averaging can be harmful. Architectures with signed relations, higher-order neighborhoods, attention, or graph rewiring may work better.

10 Graph Attention Networks (GATs)

10.1 Learning which neighbors matter

A standard GCN uses degree-based fixed normalization. A GAT instead learns a content-dependent weight for each edge in a node's neighborhood.

For node features h_i , first transform them:

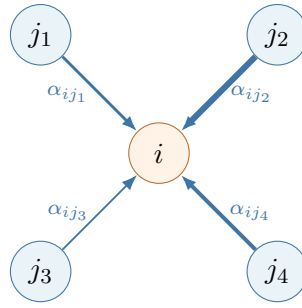
$$z_i = Wh_i.$$

Then compute an edge score, for example

$$e_{ij} = \text{LeakyReLU} \left(a^\top [z_i \| z_j] \right), \quad (36)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp(e_{ik})}, \quad (37)$$

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{ij} z_j \right). \quad (38)$$



10.2 Numerical softmax example

Worked example

Suppose one node has attention logits over self and three neighbors

$$e = [0.81, 1.85, 0.00, 1.06].$$

Applying softmax gives approximately

$$\alpha = [0.18, 0.51, 0.08, 0.23].$$

The second neighbor therefore contributes about half of the aggregated transformed feature, while the third contributes very little.

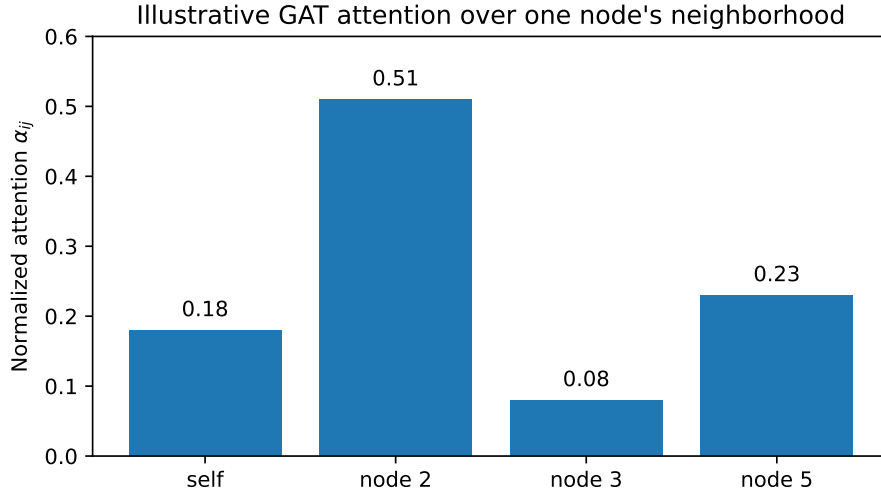


Figure 9: The same illustrative attention weights as the worked example. The weights are normalized separately for each center node’s neighborhood.

10.3 Multi-head graph attention

As in Transformers, multiple heads can be used:

$$h'_i = \parallel_{m=1}^M \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(m)} W^{(m)} h_j \right)$$

for intermediate layers, while final heads are often averaged instead of concatenated.

10.4 GAT versus Transformer attention

Both use learned attention, but the allowed interaction set differs:

- a standard GAT attends only over graph neighbors (plus often self-loops);
- a standard Transformer self-attention layer attends densely over all allowed sequence positions;
- graph Transformers may combine explicit graph structure with broader or global attention.

10.5 Interpretability caveat

GAT coefficients can be inspected, but a large attention coefficient is not automatically a causal explanation or a complete measure of feature importance. Residual paths, downstream layers, nonlinearities, and value transformations all affect the final prediction.

11 Putting the Architectures Together

11.1 Comparison table

Model	Native structure	Main routing mechanism	Long-range behavior and typical strengths
RNN	sequence	recurrent hidden state	weak for long distances; minimal sequential baseline; natural for streaming
LSTM	sequence	gated cell and hidden state	improved long-range memory; strong for moderate-size time series and sequential data
GRU	sequence	gated hidden state	similar goal to LSTM with fewer parameters; efficient recurrent baseline
Encoder-decoder	input/output pair	encode source, then decode target	depends on encoder/decoder components; natural for conditional generation and transduction
Autoencoder	vectors, images, or sequences	latent bottleneck and reconstruction	not primarily temporal; useful for representation learning, denoising, compression, and anomaly scoring
Transformer	token sequence	self-attention and cross-attention	direct pairwise access within the attention pattern; scalable contextualization and pretraining
GNN	graph	neighbor message passing	receptive field grows with graph layers; strong relational inductive bias
GAT	graph	attention-weighted neighbor messages	adaptive local neighbor importance; long-range access still requires depth or global connections

11.2 Information-flow comparison

Architecture	What is a “neighbor”?	What is carried forward?	What is learned about routing?
RNN	previous time step	hidden state	state transition weights
LSTM/GRU	previous time step	gated memory/state	gates + transition weights
Transformer	any allowed token	no recurrent state required	attention scores between token representations
GCN	graph-adjacent nodes	updated node embeddings	feature transforms; adjacency weighting mostly fixed
GAT	graph-adjacent nodes	updated node embeddings	feature transforms + edge attention weights

11.3 Choosing by problem structure

- **Short/moderate time series with limited data:** start with a strong tabular baseline, then GRU/LSTM if temporal state matters.
- **Large-scale language or long contextual sequences:** Transformer-based models are the default starting point, subject to sequence-length cost.
- **Input-output sequence mapping:** use an encoder-decoder; the components may be recurrent or Transformer-based.
- **Unsupervised compression/denoising/anomaly scoring:** autoencoders are a direct fit, but

validate that reconstruction error aligns with the real objective.

- **Explicit relational data:** GNN/GAT models exploit the graph instead of flattening relationships into ordinary features.
- **Neighbors have unequal relevance:** GAT or another adaptive message-passing layer may be more suitable than uniform/degree-based aggregation.

11.4 Data regime matters

Architecture choice should not be based only on theoretical expressiveness. Large Transformers can be statistically and computationally inefficient on small datasets. Recurrent models impose a stronger sequential bias and can be effective with less data. GNNs can gain efficiency by encoding known relational structure. Autoencoders can learn from unlabeled data, but reconstruction may emphasize nuisance variation unless the bottleneck and objective are carefully designed.

11.5 Hybrids are common

Real systems often combine these ideas:

- LSTM autoencoder for time-series anomaly detection;
- CNN or Transformer encoder plus autoregressive decoder;
- temporal GNN where each node has a recurrent history;
- graph Transformer mixing local message passing with global attention;
- encoder representations fed into a GNN for relational reasoning;
- variational sequence autoencoders with recurrent or Transformer encoders/decoders.

11.6 Training pathologies at a glance

Family	Common failure mode	Typical mitigation
RNN	vanishing/exploding gradients	gated cells, clipping, shorter BPTT
LSTM/GRU	overfitting, slow sequential training	regularization, careful hidden size, batching
Autoencoder	trivial copying / anomaly reconstruction	bottleneck, corruption, sparsity, capacity control
Transformer	memory cost, optimization instability	efficient attention, normalization, schedule, caching
GNN	over-smoothing / over-squashing	residuals, fewer layers, rewiring, improved propagation
GAT	noisy or unstable neighbor weighting	head averaging, regularization, robust graph design

12 Deeper Connections Between the Models

12.1 Gates and attention are both selective routing

An LSTM gate decides how much of a stored vector to keep or update. Attention decides how much information to retrieve from each source representation. Both are differentiable mechanisms for allocating information flow, but their granularity differs: gates usually act elementwise on one state transition, while attention allocates weights across multiple candidate sources.

12.2 Residual connections resemble memory highways

The LSTM cell update

$$c_t = f_t \odot c_{t-1} + \text{new content}$$

and a Transformer residual block

$$x_{\ell+1} = x_{\ell} + F_{\ell}(x_{\ell})$$

share an important optimization theme: preserve a relatively direct path through depth/time and learn a modification around it. They are not mathematically identical, but both reduce the burden of repeatedly reconstructing information through nonlinear transformations.

12.3 Attention as dynamic message passing

A GAT can be read as a sparse attention mechanism constrained by graph edges. A Transformer can be read as message passing on a dense graph whose nodes are tokens and whose edges are all allowed token pairs. This perspective motivates graph Transformers and sparse attention patterns.

12.4 Autoencoders and encoders are not synonymous

Every autoencoder has an encoder, but not every encoder belongs to an autoencoder. A classifier can use an encoder whose representation feeds a prediction head. A sequence-to-sequence model can use an encoder whose state feeds a decoder. The word *encoder* describes a role; *autoencoder* describes a training objective and paired architecture.

13 Practical Design Checklist

13.1 Before choosing an architecture

Ask:

1. Is the data fundamentally sequential, spatial, graph-structured, or simply tabular?
2. Does order matter, and is future context available at inference time?
3. How long are the dependencies that matter?
4. How much labeled data is available?
5. Is inference streaming/autoregressive or batch/offline?
6. Is there a known graph, or would graph edges need to be learned/constructed?
7. Does the objective require reconstruction, prediction, generation, or representation learning?

13.2 Sequence-model checklist

For RNN/LSTM/GRU models, specify hidden width, number of layers, directionality, whether hidden state is reset between examples, truncation length for BPTT, dropout placement, and whether sequence lengths are masked correctly.

For Transformers, specify model width, number of heads, depth, feed-forward expansion, positional method, attention mask, maximum context length, normalization placement, dropout, and whether inference uses key/value caching.

13.3 Graph-model checklist

Specify graph construction, edge directionality, self-loops, edge features, aggregation rule, number of message-passing layers, readout for graph-level tasks, treatment of isolated nodes, and whether train/validation/test splitting leaks information through the graph topology.

13.4 Evaluation discipline

A more complex architecture should beat a simple baseline under the *same* split and metric. Compare parameter count, training time, inference cost, and robustness, not only peak validation score. For temporal data, preserve time ordering. For graphs, choose splits that match deployment: random node splits, temporal link splits, and inductive unseen-graph splits answer different questions.

14 Compact Mathematical Cheat Sheet

Vanilla RNN

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \quad y_t = W_y h_t + b_y.$$

LSTM

$$\begin{aligned} f_t &= \sigma(W_f u_t + b_f), \quad i_t = \sigma(W_i u_t + b_i), \\ \tilde{c}_t &= \tanh(W_c u_t + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ o_t &= \sigma(W_o u_t + b_o), \quad h_t = o_t \odot \tanh(c_t). \end{aligned}$$

GRU

$$\begin{aligned} z_t &= \sigma(W_z x_t + U_z h_{t-1}), \quad r_t = \sigma(W_r x_t + U_r h_{t-1}), \\ \tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1})), \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \end{aligned}$$

Autoencoder / VAE

$$\begin{aligned} z &= f_\theta(x), \quad \hat{x} = g_\phi(z), \quad \mathcal{L}_{AE} = \|x - \hat{x}\|^2, \\ \mathcal{L}_{ELBO} &= \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{\text{KL}}(q(z|x) \| p(z)). \end{aligned}$$

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

GCN

$$H^{(\ell+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(\ell)} W^{(\ell)}\right), \quad \tilde{A} = A + I.$$

GAT

$$\begin{aligned} e_{ij} &= \text{LeakyReLU}(a^\top [W h_i \| W h_j]), \quad \alpha_{ij} = \text{softmax}_{j \in \mathcal{N}(i)}(e_{ij}), \\ h'_i &= \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j\right). \end{aligned}$$

Key point

If you remember only one organizing principle: **RNN/LSTM/GRU are state-transition models, encoder-decoder describes a directional architecture, autoencoders learn through reconstruction, Transformers perform attention-based token interaction, GNNs perform graph-constrained message passing, and GATs make that graph aggregation attention-weighted.**

Selected References and Further Reading

References

- [1] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory.” *Neural Computation*, 1997.
- [2] K. Cho et al. “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.” 2014.
- [3] I. Sutskever, O. Vinyals, and Q. Le. “Sequence to Sequence Learning with Neural Networks.” 2014.
- [4] D. Bahdanau, K. Cho, and Y. Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate.” 2014/2015.
- [5] G. Hinton and R. Salakhutdinov. “Reducing the Dimensionality of Data with Neural Networks.” *Science*, 2006.
- [6] D. Kingma and M. Welling. “Auto-Encoding Variational Bayes.” 2013/2014.
- [7] A. Vaswani et al. “Attention Is All You Need.” 2017.
- [8] T. Kipf and M. Welling. “Semi-Supervised Classification with Graph Convolutional Networks.” 2016/2017.
- [9] P. Veličković et al. “Graph Attention Networks.” 2017/2018.